

基于 IOH 的子模优化实验报告

基于 IOH 的子模优化实验报告

1. 实验目标

本实验围绕 IOH 框架中的两类 0-1 子模优化问题展开：

- MaxCut，问题编号为 2000-2004
- MaxCoverage，问题编号为 2100-2127

实验的直接目标有两层。第一层是完成课程要求中的基础演化算法实现，包括二进制编码、种群、自然选择、bit-wise mutation 和 uniform crossover 等基本组成。第二层是在统一实验框架下，对两类问题进行系统实验，记录每个实例上的最终结果，并结合平均 fitness 曲线分析算法效果。

这次工作的一个重要特点是：项目不是停留在“做出一版能跑的 GA”之后就结束，而是在完整保留基础 EA 实现的前提下，继续根据问题结构做了分流优化。我最终形成的方案是：

- MaxCoverage 保留并强化了 binary EA 主线
- MaxCut 在完成基础 EA 实现后，正式结果改为更适合该问题的固定顺序 1-flip simulated annealing

这份报告既覆盖基础 EA 的实现要求，也真实反映了从统一基线到最终方案的探索过程。

2. 从共享结构到问题分流

这次项目最开始坚持的思路是：能共享的部分先尽量共享，不急着一上来就把两类问题完全拆开。于是我先搭了一套统一的 binary EA 框架，同时处理 MaxCut 和 MaxCoverage。这一阶段已经具备了课程要求中的基本组成：

- 0-1 向量表示解

- 随机初始化
- tournament selection
- uniform crossover
- bit-wise mutation
- elitism
- 固定 fitness evaluation budget
- 多次独立运行后统计平均结果并出图

这套共享结构的价值主要有两点。第一，它先把课程要求中的基础实验框架完整搭起来了。第二，它让后面的每一次修改都能放在同一条实验流程里比较，不会变成零散的小试验。

但实验继续往下做以后，我逐渐发现：共享实验框架是对的，共享求解思路却不一定对。MaxCut 和 MaxCoverage 虽然都可以用二进制编码来表示解，但它们真正困难的地方并不一样。

对 MaxCoverage 来说，最核心的问题是约束。很多解表面上 fitness 不低，但其实并不可行。如果仍然只按 fitness 排序，这些解会把种群方向带偏，连历史最优也可能被误导。所以后面的调整，主要都围绕“先把搜索尽量留在可行域附近”展开，包括：

- 记录每个解的 fitness、violation 和 penalty
- 比较解时优先保留可行解，而不是只看 fitness
- 对明显不可行的初始解和子代做简单 repair

对 MaxCut 来说，问题则完全不同。它没有约束，所有二进制解都可以直接比较，因此没有必要花大量精力处理“可行还是不可行”。它更需要的是稳定的局部改进能力。后面的实验说明，如果继续围绕原来的 population-GA 一直加机制，提升并不稳定；相比之下，改用固定顺序 1-flip simulated annealing 后，结果更稳，曲线也更平滑。

之所以最后选 1-flip，是因为它很符合 MaxCut 本身的结构。对这个问题来说，一个二进制位就对应一个顶点被放在哪一侧，而把某一位翻转一次，本质上就是把一个顶点移到 cut 的另一边。这样的改动很小、很直接，而且它带来的收益也比较容易理解，因为真正受影响的主要是和这个顶点相连的边。换句话说，MaxCut 很适合用“先看翻动一个点会不会更好”的局部搜索直觉来做。

同时，1-flip 也符合一个很自然的搜索想法：如果某次单点翻转能让 cut 变大，就立即接受；如果暂时变差，也可以在前期以小概率接受，从而避免太早卡在局部最优。这正是 simulated annealing 的优势。它既保留了局部搜索简单直接的优点，又给了算法一定的跳出能力，所以最后比继续硬撑在统一的 population-GA 上更合适。

所以，项目最后保留下来的不是“一套算法解决所有问题”，而是“共享实验框架，但在求解器层面按问题分流”：

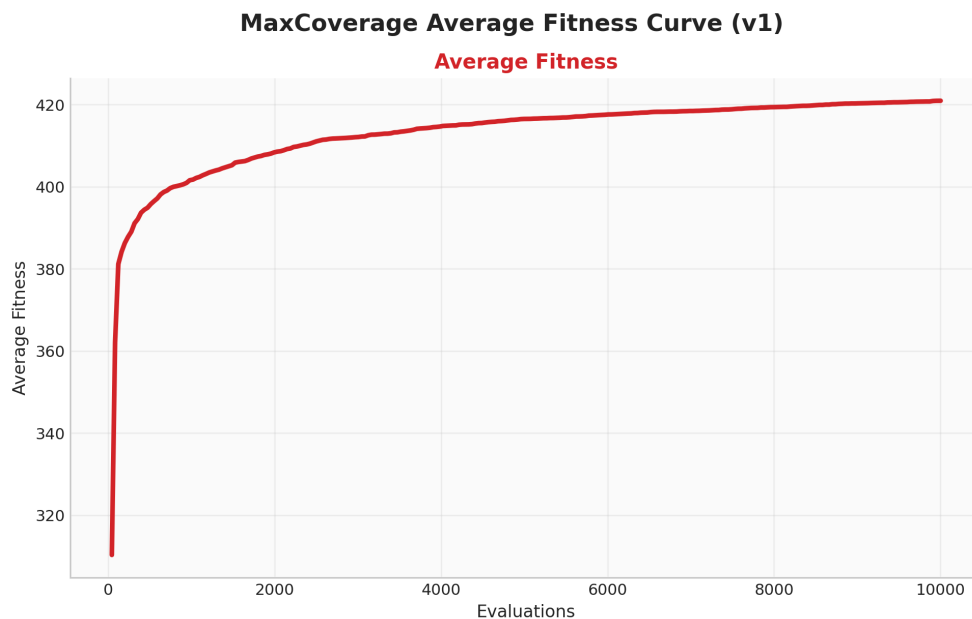
- 共享的是编码方式、实验流程、统计方法和出图方式
- 分流的是真正执行搜索的策略

遗憾的探索

这一过程中也有一些没有保留下来的尝试。它们最后没有成为正式方案，但并不是白做，因为正是这些失败，让我更快看清了问题的边界。

在 MaxCoverage 上，我曾尝试把 violation 直接乘一个权重，加到选择分数里，希望用一个统一分数同时处理“解有多好”和“解是否越界”。这个思路看起来很直接，但实际效果并不好：权重不容易调稳，而且会把一些本来有潜力的解也一起压下去。

下面这张图是 MaxCoverage 更早期、效果也更弱的一版正式归档结果，对应的预算同样是 10000 evaluations，可以看出当时这条思路还没有带来理想的提升效果。

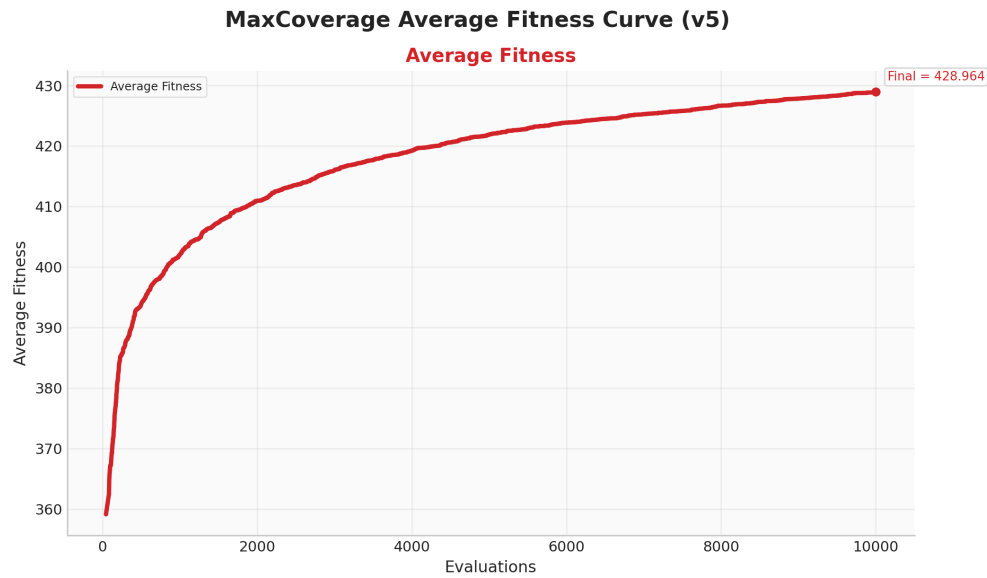


MaxCoverage 早期试错图 1

我也试过更贪心的约束感知初始化，希望在一开始就构造出更好的可行解。这个方向听起来合理，但它会增加初始化开销，还会削弱种群的多样性，结果是后续搜索反而不够灵活。

另外，我还尝试过更激进的变异率、可行优先的父代选择，以及在 repair 之后再接一小段局部搜索。这些改动单独看都有道理，但综合结果并不稳定，所以最后都撤回了。

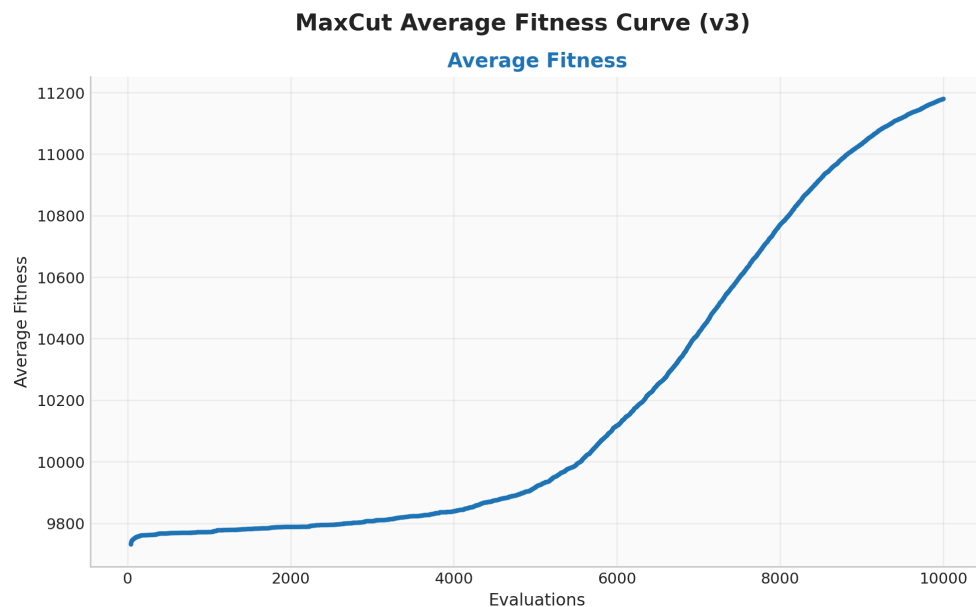
下面这张图对应 MaxCoverage 另一版没有被最终保留的 10000 evaluations 正式结果。它代表的不是“完全没进步”，而是“试过以后发现还不够稳，也不够值得作为最终方案保留”。



MaxCoverage 早期试错图 2

在 MaxCut 上，我也试过继续围绕 population-GA 做增强，希望在不分流的前提下把效果继续推高。但多轮实验下来，这条线始终没有形成足够稳定的提升。此外，我还尝试过随机选择 bit 位进行翻转，希望用更随机的方式增加搜索范围，但效果并不好，提升不够稳定。最后保留下来的做法，反而是更简单也更贴合问题结构的固定顺序 1-flip，也就是按固定顺序逐个尝试翻转顶点对应的 bit。

下面这张图是 MaxCut 在固定顺序 1-flip 完全定下来之前的一版 10000 evaluations 结果。它能说明，随机翻转或者邻域结构还没有完全理顺时，曲线表现并不如最终方案稳定。



MaxCut 早期试错图

回头看，这一节最重要的收获不是某一次修改立刻成功，而是慢慢弄清楚了什么应该共享，什么必须分开。算法改进很少是一条笔直向前的路，更多时候是先尝试、再回退、再调整。进步其实就是走两步退一步的螺旋上升过程。

3. 算法设计

3.1 基础 EA 实现

课程要求中的基础 EA 已经完整实现，主要设计如下。

解的表示

每个候选解都用一个长度为 n 的 0-1 向量表示。向量中的每一位都对应一个二元决策，这就是标准的 binary representation。

初始化

初始解按概率随机生成。也就是说，每一位都以 `one_prob` 的概率取值为 1。

自然选择

父代选择采用 tournament selection。每次从当前种群中随机抽取若干个体，选出其中更好的个体作为父代。生存者选择采用 elitism，也就是每一代先保留当前最好的若干个体，再补充新生成的子代。

交叉与变异

- 交叉算子采用 uniform crossover
- 变异算子采用 bit-wise mutation

uniform crossover 对每一位独立决定来自父代 A 还是父代 B；bit-wise mutation 则以 $p_m = 1/n$ 的默认概率独立翻转每一位。

停止条件

实验采用固定 fitness evaluation budget，而不是固定迭代代数。因为不同问题、不同搜索策略下，每一代真正消耗的评估次数并不完全一样，用 budget 作为统一标准会更方便公平比较。

3.2 MaxCoverage 的最终方案

MaxCoverage 的正式版本保留了 binary EA 主线，但围绕可行性主要做了三点调整。

第一，初始化采用按维度自适应的稀疏采样。因为 MaxCoverage 的可行区域比较窄，如果初始解过密，算法一开始就很容易掉进重惩罚区。当前正式设置中：

- $n = 450$ 时，目标初始 1 的个数约为 4.5，对应 $\text{one_prob} = 0.0100$
- $n = 595$ 时，目标初始 1 的个数约为 4.0，对应 $\text{one_prob} \approx 0.00672$
- $n = 760$ 时，目标初始 1 的个数约为 3.0，对应 $\text{one_prob} \approx 0.00395$

第二，比较规则从“只看 raw fitness”改成了 feasible-first。也就是说：

- 先判断一个解是否可行
- 可行解优先于不可行解
- 在可行解内部再比较 fitness
- 在不可行解内部再参考 penalty 和 violation

第三，对不可行解做轻量级 repair。这个修复不是复杂的局部搜索，而是在已经取 1 的位置里抽样若干候选位，尝试把其中某一位从 1 改成 0，再根据同一套 feasible-first 规则选出更好的修复版本。这个 repair 既用于初始化阶段，也用于子代生成之后。

除了这些和可行性直接相关的改动，MaxCoverage 的种群规模也是我后期重点调过的一组参数。正式压缩实验里，我先把种群数量从 20 调到 10；在个人进一步调试时，我又继续往下试到 5，感觉效果还在变好；最后再收缩到 2，整体表现反而更集中，也更符合这个问题“把有限预算尽量集中在少量高质量搜索轨迹上”的直觉。

后来我再去查相关文献时，也注意到 (1+1) EA 这一类极小规模搜索在子模优化问题上是有不错表现的。这让我觉得，自己沿着 20 -> 10 -> 5 -> 2 这样一路收缩种群的探索，实际上已经摸到了一个比较合理的方向。只是因为时间有限，我没有再继续沿着“是否进一步贴近 (1+1) EA”这条线做更系统的扩展实验，所以最终方案和文献中的做法是接近，但并不完全相同。

这一套设计的核心想法可以概括成一句话：对 MaxCoverage 来说，先尽量把搜索放在可行域附近，再考虑怎样把 fitness 做高。

3.3 MaxCut 的最终方案

MaxCut 在项目早期也使用过统一的 binary GA，所以基础 EA 要求在这个问题族上同样已经实现并验证过。但在最终正式版本中，MaxCut 被分流到固定顺序 1-flip simulated annealing。

这样做的原因很直接：MaxCut 没有约束，问题的关键不在于“如何处理不可行解”，而在于“如何稳定地做局部改进，并在需要时跳出局部最优”。与其继续在 population-GA 上不断加机制，不如采用更贴近它结构的单解搜索。

正式版本中的 MaxCut 求解流程如下：

- 从一个随机二进制解开始
- 依次尝试翻转每个顶点对应的比特
- 若邻域解更优则直接接受
- 若邻域解更差，则在温度较高时按模拟退火准则以一定概率接受
- 随温度下降，搜索逐渐从带随机性的探索转向近似爬山式改进

这说明，项目最终采用的是“共享同一套实验框架，但在 solver 层按问题分流”的思路。这种分流不是对基础 EA 的否定，而是在完成基础实现以后，根据实验结果做出的更合适选择。

3.4 统计与出图方式

每个问题实例独立运行 10 次。由于不同 run 的代数和评估次数不一定能一一对上，正式统计采用“按 evaluations 对齐，再 carry-forward 平均”的方式生成平均曲线。这样可以避免直接按代数下标平均带来的错位问题，让曲线更真实地反映固定预算下的整体表现。

4. 代码实现

4.1 为什么这样组织代码

在代码实现上，我没有把所有逻辑堆在一个长脚本里，而是从一开始就尽量按“职责”来拆。这样做的好处很直接：一方面，每一块代码只负责一件相对明确的事，读起来更清楚；另一方面，后面要改某个算法细节时，通常只需要改对应的一小块，而不容易把整条实验流程一起弄乱。

这次项目里，真正经常变化的是求解策略，比如初始化该不该更稀疏、MaxCoverage 要不要 repair、MaxCut 最后要不要从 GA 分流出去。相对稳定的则是实验流程本身，比如加载实例、记录 history、批量重复运行、汇总结果、出图。

因此我的组织思路是：把容易变化的部分放在求解器和 phase 内部，把不太变化的部分留在统一实验框架里。这样后面即使算法改动比较大，整套实验管线也还能保持稳定。

除了主体算法代码，我还借助 AI 协助生成和整理了多份脚本，把运行、记录和出图串成了一条自动化流程。具体来说，我把 Makefile、批量实验脚本和若干 Markdown 记录工具组织到了一起，让自己可以一键运行正式组，也可以一键运行测试组；运行结束后，程序会自动完成批量求解、生成图片、保存结果，并把关键参数变化和实例结果记录下来。

这样做带来的好处不只是省时间。因为每次实验都有对应的归档、图像和文字记录，所以我后面在回看某一版结果时，更容易知道“这一版改了什么、为什么改、结果变成了什么样”。这让开发、检索、测试和回溯都方便了很多，也让整个项目从“能跑”变成了“好维护、好比较、好复现”。

4.2 六阶段结构

整个 binary EA 主体被我拆成了六个 phase，这样代码结构和算法流程基本是一一对应的。

第一阶段负责 setup。这里主要完成问题实例加载、随机数种子设置、维度读取、默认参数整理，以及对一些参数做合法化处理。它对应的是“实验开始前先把问题和参数准备好”这一步。

第二阶段负责 population。这里处理二进制解的采样、初始种群生成、解的评估，以及当前最优个体的记录。对 MaxCoverage 来说，这一层还顺带承接了初始化 repair 和可行性信息维护，所以它对应的是“编码、初始化、评估、初始最优记录”这一整块算法思路。

第三阶段负责 selection。这里专门实现父代选择，也就是 tournament selection。把它单独拆出来的好处是，后面如果我要改选择逻辑，不需要去动交叉、变异或者统计部分。

第四阶段负责 variation。这里集中放交叉和变异，也就是 uniform crossover 和 bit-wise mutation。这样算法里“如何产生新个体”的部分就被单独隔离出来了，逻辑比较清楚。

第五阶段负责 survival。这里处理生存者保留，也就是 elitism。到了 MaxCoverage 的正式版本后，这一层又自然承接了 feasible-first 的排序规则，所以它对应的是“哪些个体应该留下来进入下一代”。

第六阶段负责 reporting。这里不再关心怎样搜索，而是专门负责停止判断、history 记录、结果打包、CSV 输出，以及多次运行后的平均统计。它直接对应实验报告里的结果汇总和曲线生成需求。

这样拆下来以后，算法流程本身就 and 代码结构自然对齐了：setup、初始化、选择、变异、保留、记录，读代码时基本不用反复跳来跳去。

如果从工程组织的角度再往下看，主要 Python 文件和它们的职责关系也比较清楚：

主要文件	主要作用	对应的算法或实验思路
main.py	单次求解入口，负责把各阶段串起来，并在 MaxCut 与 MaxCoverage 之间分流	主求解流程；MaxCut 的专门 solver；MaxCoverage 的 repair 调度
ea_phase1_setup.py	负责加载实例、读取维度、整理默认参数	setup 阶段；默认初始化概率；默认变异率
ea_phase2_population.py	负责解采样、初始种群生成、评估、记录当前最优	binary representation；初始化；可行性信息维护
ea_phase3_selection.py	负责父代选择	tournament selection
ea_phase4_variation.py	负责交叉与变异	uniform crossover；bit-wise mutation
ea_phase5_survival.py	负责保留进入下一代的个体	elitism；MaxCoverage 的 feasible-first 保留规则
ea_phase6_reporting.py	负责停止判断、history、结果打包和多次运行后的平均	实验记录；结果汇总；曲线统计
batch_run.py	负责批量跑所有实例并输出汇总结果	正式组/测试组批量实验；实例级结果统计
plot_curves.py	负责从汇总数据生成曲线图	平均 fitness 曲线；正式出图
record_formal_run.py	负责记录正式版本参数变化	版本归档；实验可追溯性
write_instance_markdown.py	负责把每个实例的最终结果整理成 Markdown	报告附表；实例结果留档

4.3 MaxCut 如何脱离主体框架

MaxCut 最终没有继续留在 population-based EA 主体里，而是分流到了固定顺序 1-flip simulated annealing。但这里我并没有把它完全写成另一套无关的程序，而是让它只在“真正执行搜索”的那一层分开，其他部分尽量继续复用原来的实验框架。

MaxCut 虽然不再走“种群初始化、父代选择、交叉变异、生存者保留”这条主线，但它仍然共享：

- 同样的实例加载方式
- 同样的随机初始化入口
- 同样的 fitness 评估接口
- 同样的 history 记录格式
- 同样的单次运行结果打包方式
- 同样的批量实验、结果汇总和出图流程

这样做的好处是，MaxCut 虽然在算法上分流了，但在实验层面仍然和 MaxCoverage 保持统一。

4.4 代码结构与算法思路的对应关系

- 二进制编码、随机初始化、初始种群生成，对应的是 population 阶段。
- tournament selection，对应的是 selection 阶段。
- uniform crossover 和 bit-wise mutation，对应的是 variation 阶段。
- elitism 和 MaxCoverage 的 feasible-first 保留规则，对应的是 survival 阶段。
- MaxCoverage 的稀疏初始化、可行性判断和 repair，主要落在 setup、population 和主调度逻辑之间。
- MaxCut 的固定顺序 1-flip simulated annealing，则作为分流后的专门 solver 存在，但仍然接回统一的 reporting 和批量实验流程。
- 多次独立运行后的平均曲线、每个实例的最终结果汇总，以及报告所需的 CSV 和图片输出，则集中落在 reporting 和批量实验部分。

从可维护性的角度看，我觉得这套组织方式最重要的一点是：算法思路不是只写在报告里，而是能在代码结构里直接看出来。即使不先看完整个项目，也大致能从模块划分上看出“哪一块在解决什么问题”。这让后续修改、调试和扩展都容易很多。

5. 实验设置

正式参数如下。

项目	设置
问题族	MaxCut + MaxCoverage
总预算	每个实例每次运行 10000 次 fitness evaluations
重复次数	每个实例 10 次独立运行
基础种群大小	2
交叉概率	0.5
变异率	默认 1/n
锦标赛大小	2
精英个数	1
横轴	evaluations
曲线纵轴	average best fitness
MaxCut 初始化	one_prob = 0.5
MaxCoverage 初始化	按维度自适应稀疏初始化

这里需要补充说明一点：MaxCut 的最终正式结果采用的是单解模拟退火，因此形式上保留的 pop_size、crossover_rate、mutation_rate 字段，主要是为了维持统一接口和实验记录；真正驱动搜索的，是固定顺序 1-flip simulated annealing 本身。

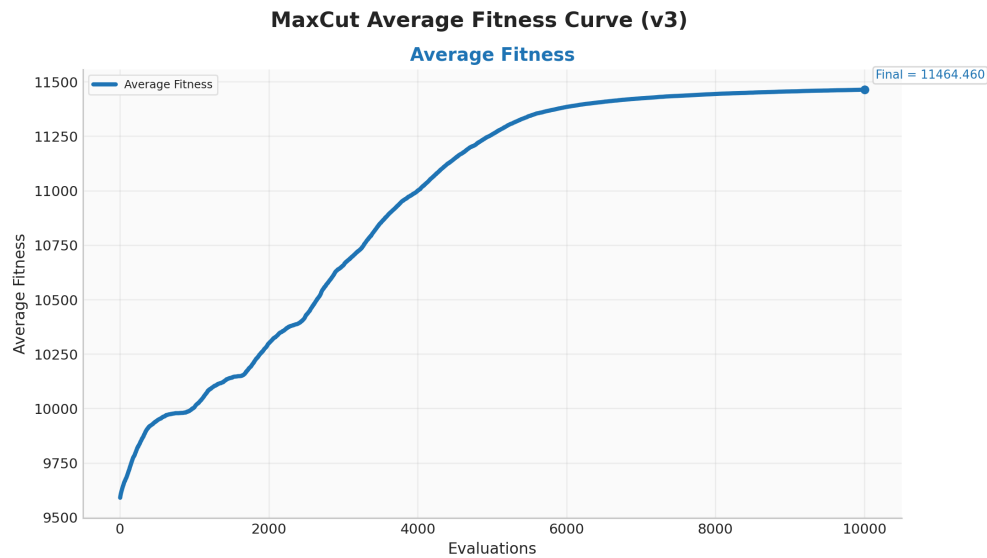
问题规模方面：

- MaxCut 共 5 个实例，维度均为 800
- MaxCoverage 共 28 个实例，维度分别为 450、595、760

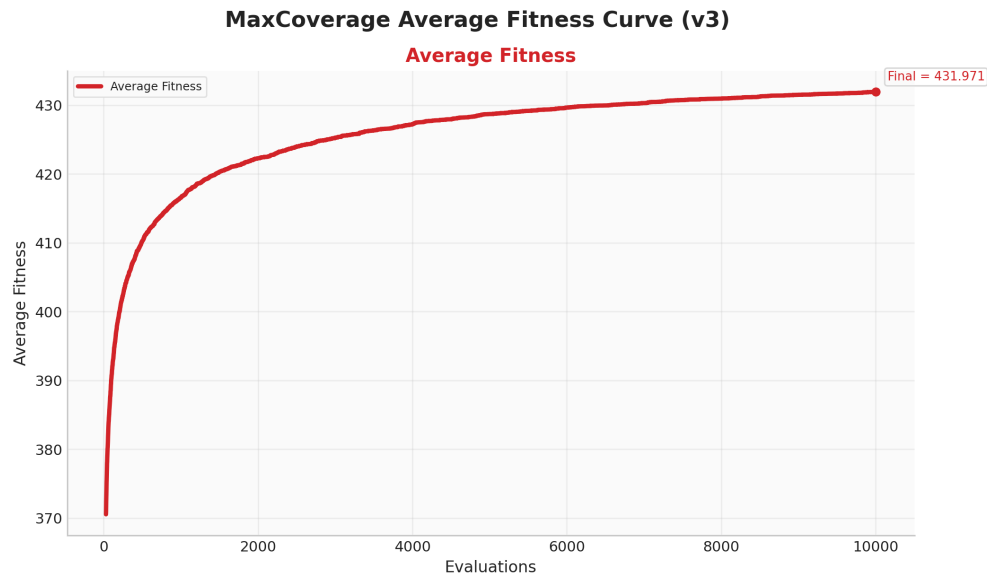
6. 结果与分析

6.1 曲线结果

两类问题的正式曲线如下：



MaxCut 正式曲线



MaxCoverage 正式曲线

从曲线形状上看，MaxCut 的提升过程比较平滑，前期上升较快，后期逐步趋稳，说明分流后的单解搜索在固定预算下表现比较稳定。相比之下，MaxCoverage 的曲线虽然前期也能明显上升，但后期波动更大，这和它的可行区域较窄、不同实例难度差异明显有关。

6.2 整体表现

按实例平均后的整体表现可以概括为：

- MaxCut 的 `average_best_fitness` 实例均值为 11464.460
- MaxCoverage 的 `average_best_fitness` 实例均值为 431.971

对 MaxCut 来说，五个实例之间的表现非常接近，说明最终方案比较稳定。其中：

- 最好的是 2003, `average_best_fitness` = 11498.700
- 相对最低的是 2002, `average_best_fitness` = 11446.600

对 MaxCoverage 来说，不同实例之间的差异明显更大。其中：

- 最好的是 2103, `average_best_fitness` = 707.400
- 相对最难的是 2125, `average_best_fitness` = 325.600

另外，MaxCoverage 有 4 个实例的 `average_final_mean_fitness` 为负，分别是 2108、2109、2110、2111。这并不表示算法没有找到过好解，而是说明在这些更困难的实例上，运行结束时种群整体质量还不够稳定。

把这两个指标放在一起看，可以更完整地说明 MaxCoverage 的难点：好的个体可以找到，但想让整个种群长期稳定地停在高质量可行区域，仍然更难。

6.3 结果讨论

从这次实验可以得到两个比较清楚的结论。

第一，统一的 binary EA 是一个必要且有效的起点。它完成了课程要求中的核心实现，也给后续比较提供了统一框架。

第二，问题结构会决定“基础 EA 之后该往哪里改”。对 MaxCoverage 来说，关键是把可行性真正纳入搜索主逻辑，因此 `feasible-first + repair + sparse initialization` 是有效方向；对 MaxCut 来说，关键是局部改进和跳出局部最优，因此最终转向 1-flip simulated annealing 更自然。

也就是说，本实验最重要的收获是逐步确认了：

- 哪些结构应该共享
- 哪些部分必须按问题族分流

7. 各问题实例最终结果

7.1 MaxCut

Problem ID	Problem Name	n	Runs	Budget	Avg Best	Std Best	Min Best	Max Best
2000	MaxCut2000	800	10	10000	11451.900	27.848	11392	11498.700
2001	MaxCut2001	800	10	10000	11462.300	22.619	11430	11498.700

Problem ID	Problem Name	n	Runs	Budget	Avg Best	Std Best	Min Best	Max Best
2002	MaxCut2002	800	10	10000	11446.600	40.158	11407	1151
2003	MaxCut2003	800	10	10000	11498.700	33.209	11445	1155
2004	MaxCut2004	800	10	10000	11462.800	30.521	11406	1153

7.2 MaxCoverage

Problem ID	Problem Name	n	Runs	Budget	Avg Best	Std Best	Min Best	Max Best
2100	MaxCoverage2100	450	10	10000	436.300	2.193	433	440
2101	MaxCoverage2101	450	10	10000	438.100	3.807	432	445
2102	MaxCoverage2102	595	10	10000	558.700	6.357	547	570
2103	MaxCoverage2103	760	10	10000	707.400	5.696	698	720
2104	MaxCoverage2104	450	10	10000	364.600	5.122	357	375
2105	MaxCoverage2105	450	10	10000	365.100	4.571	359	375
2106	MaxCoverage2106	595	10	10000	408.300	3.407	402	415
2107	MaxCoverage2107	760	10	10000	435.700	3.770	430	445
2108	MaxCoverage2108	450	10	10000	395.900	7.582	380	410
2109	MaxCoverage2109	450	10	10000	386.100	5.243	377	395
2110	MaxCoverage2110	595	10	10000	431.100	7.556	416	445
2111	MaxCoverage2111	760	10	10000	431.200	12.766	397	445
2112	MaxCoverage2112	450	10	10000	406.100	4.742	400	415
2113	MaxCoverage2113	450	10	10000	409	5.477	401	415
2114	MaxCoverage2114	595	10	10000	507.100	5.088	498	520
2115	MaxCoverage2115	760	10	10000	641	5.441	633	655
2116	MaxCoverage2116	450	10	10000	334	3.066	329	345
2117	MaxCoverage2117	450	10	10000	335.600	3.720	331	345
2118	MaxCoverage2118	595	10	10000	363.200	5.016	355	375
2119	MaxCoverage2119	760	10	10000	388.500	4.031	384	395
2120	MaxCoverage2120	450	10	10000	406.100	4.742	400	415

Problem ID	Problem Name	n	Runs	Budget	Avg Best	Std Best	Min Best	Max Best
2121	MaxCoverage2121	450	10	10000	409	5.477	401	417
2122	MaxCoverage2122	595	10	10000	507.100	5.088	498	516
2123	MaxCoverage2123	760	10	10000	641	5.441	633	649
2124	MaxCoverage2124	450	10	10000	326.900	2.625	321	332
2125	MaxCoverage2125	450	10	10000	325.600	3.292	322	329
2126	MaxCoverage2126	595	10	10000	356.200	3.458	349	363
2127	MaxCoverage2127	760	10	10000	380.300	3.494	375	385

8. 结论

本实验已经完整实现并验证了课程要求中的基础演化算法框架，包括 binary representation、种群、自然选择、uniform crossover 和 bit-wise mutation。在这个基础上，又通过多轮实验形成了更贴合问题结构的最终方案：

- MaxCoverage 走向可行优先的 binary EA
- MaxCut 走向固定顺序 1-flip simulated annealing

从最终结果看，这种分流是合理的。MaxCut 表现稳定，说明单解局部搜索更适合它的结构；MaxCoverage 虽然仍然存在难实例和种群稳定性问题，但 feasible-first + repair + sparse initialization 已经明显比“只看 raw fitness 的统一 GA”更符合这个问题的约束特点。

如果后续还要继续改进，最值得继续投入的方向依然是 MaxCoverage，尤其是进一步提高困难实例上的终止种群质量，而不是只提升 best-so-far 个体。

9. 参考文献

1. F. Neumann, A. Neumann, C. Qian, V. A. Do, J. de Nobel, D. Vermetten, S. S. Ahouei, F. Ye, H. Wang, T. Bäck. *Benchmarking Algorithms for Submodular Optimization Problems Using IOHProfiler*. CoRR abs/2302.01464, 2023.